Travaux Pratiques N°4

Architecture Orientee Services

(SOA — Service-Oriented Architecture)

Module : Architectures Logicielles
Encadrant : Pr. BE ELBAGHAZAOUI

Realise par : ilyas machhoul

Filiere : IACS

Annee universitaire : 2025/2026

Table des matières

1 Etape 1 — Eureka Server **2**
1.1 Creation du projet 2
1.2 Configuration `application.properties` 2
1.3 Classe principale 2

2 Etape 2 — Product Service **3**
2.1 Configuration 3
2.2 Modele de donnees — `Product.java` 3
2.3 Repository 3
2.4 Service — `ProductService.java` 3
2.5 Controleur REST — `ProductController.java` 4
2.6 Classe principale 4

3 Etape 3 — Order Service **5**
3.1 Configuration 5
3.2 Modele — `Order.java` 5
3.3 Client Feign — `ProductClient.java` 5
3.4 DTO — `ProductDTO.java` 6
3.5 Service — `OrderService.java` 6
3.6 Controleur — `OrderController.java` 6
3.7 Classe principale 7

4 Exercices pratiques **8**
4.1 Exercice 1 — Mise a jour du stock 8
4.2 Exercice 2 — API Gateway 8
4.3 Exercice 3 — Gestion des erreurs 9

5 Conclusion **10**
5.1 Tableau de bord Eureka Server 10
5.2 Test 1 — Creation d'un produit (POST) 11
5.3 Test 2 — Liste des produits (GET) 11
5.4 Test 3 Créer une commande (POST) 12

Chapitre 1

Etape 1 — Eureka Server

1.1 Creation du projet

Configuration Spring Initializr

Project	Maven
Language	Java
Spring Boot	3.2.x
Group	`com.ecommerce`
Artifact	`eureka-server`
Dependency	`Eureka Server`

1.2 Configuration `application.properties`

```
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Listing 1.1 – application.properties — Eureka Server

1.3 Classe principale

```java
package com.ecommerce.eurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Listing 1.2 – EurekaServerApplication.java

Apres demarrage (`mvn spring-boot:run`), acceder au tableau de bord : `http://localhost:8761`.

Chapitre 2

Etape 2 — Product Service

2.1 Configuration

```
server.port=8081
spring.application.name=product-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

Listing 2.1 – application.properties — Product Service

2.2 Modele de donnees — `Product.java`

```
package com.ecommerce.productservice.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

Listing 2.2 – Product.java

2.3 Repository

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> { }
```

Listing 2.3 – ProductRepository.java

2.4 Service — `ProductService.java`

```java
@Service
@RequiredArgsConstructor
public class ProductService {
    private final ProductRepository productRepository;

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
    public Optional<Product> getProductById(Long id) {
        return productRepository.findById(id);
    }
    public Product createProduct(Product product) {
        return productRepository.save(product);
    }
    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

Listing 2.4 – ProductService.java

2.5 Controleur REST — `ProductController.java`

```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.getProductById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

Listing 2.5 – ProductController.java

2.6 Classe principale

```java
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Listing 2.6 – ProductServiceApplication.java

Chapitre 3

Etape 3 — Order Service

3.1 Configuration

```
server.port=8082
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

Listing 3.1 – application.properties — Order Service

3.2 Modele — `Order.java`

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status; // PENDING, CONFIRMED, SHIPPED, DELIVERED
}
```

Listing 3.2 – Order.java

3.3 Client Feign — `ProductClient.java`

```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
}
```

Listing 3.3 – ProductClient.java

3.4 DTO — ProductDTO.java

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

Listing 3.4 – ProductDTO.java

3.5 Service — OrderService.java

```java
@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductClient productClient;

    public List<Order> getAllOrders() {
        return orderRepository.findAll();
    }
    public Order createOrder(Long productId, Integer quantity) {
        ProductDTO product = productClient.getProductById(productId);
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        return orderRepository.save(order);
    }
}
```

Listing 3.5 – OrderService.java

3.6 Controleur — OrderController.java

```java
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @GetMapping
    public List<Order> getAllOrders() {
        return orderService.getAllOrders();
    }
    @PostMapping
    public Order createOrder(@RequestParam Long productId,
                             @RequestParam Integer quantity) {
        return orderService.createOrder(productId, quantity);
    }
}
```

Listing 3.6 – OrderController.java

3.7 Classe principale

```java
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Listing 3.7 – OrderServiceApplication.java

Chapitre 4

Exercices pratiques

4.1 Exercice 1 — Mise a jour du stock

```java
@PutMapping("/{id}/stock")
public ResponseEntity<Product> updateStock(
        @PathVariable Long id,
        @RequestParam Integer quantity) {
    return productService.updateStock(id, quantity)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

Listing 4.1 – PUT /api/products/{id}/stock — ProductController.java

```java
public Optional<Product> updateStock(Long id, Integer quantity) {
    return productRepository.findById(id).map(p -> {
        if (p.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant");
        }
        p.setStock(p.getStock() - quantity);
        return productRepository.save(p);
    });
}
```

Listing 4.2 – Logique metier — ProductService.java

4.2 Exercice 2 — API Gateway

```yaml
server:
  port: 8080
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/products/**
          filters:
            - RewritePath=/products/(?<segment>.*), /api/products/${segment}
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/orders/**
          filters:
            - RewritePath=/orders/(?<segment>.*), /api/orders/${segment}
eureka:
  client:
    service-url:
```

```
      defaultZone: http://localhost:8761/eureka/
```

Listing 4.3 – application.yml — API Gateway

4.3 Exercice 3 — Gestion des erreurs

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(FeignException.NotFound.class)
    public ResponseEntity<Map<String,String>> handleProductNotFound(
            FeignException.NotFound ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of("error", "Produit introuvable"));
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String,String>> handleRuntime(
            RuntimeException ex) {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
                .body(Map.of("error", ex.getMessage()));
    }
}
```

Listing 4.4 – GlobalExceptionHandler.java

Chapitre 5

Conclusion

Au cours de ce TP, nous avons mis en place une architecture microservices complete basee sur les principes SOA. Les points cles acquis sont :

— Registre de services avec **Eureka** pour la decouverte dynamique.

— Services REST autonomes avec **Spring Boot** et **Spring Data JPA**.

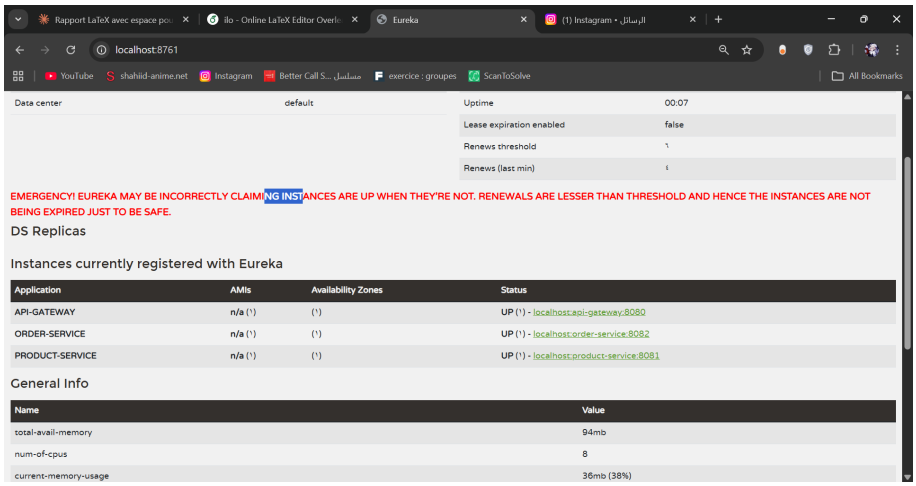
— Communication declarative inter-services via **OpenFeign**.

— Routage centralise via une **API Gateway** Spring Cloud Gateway.

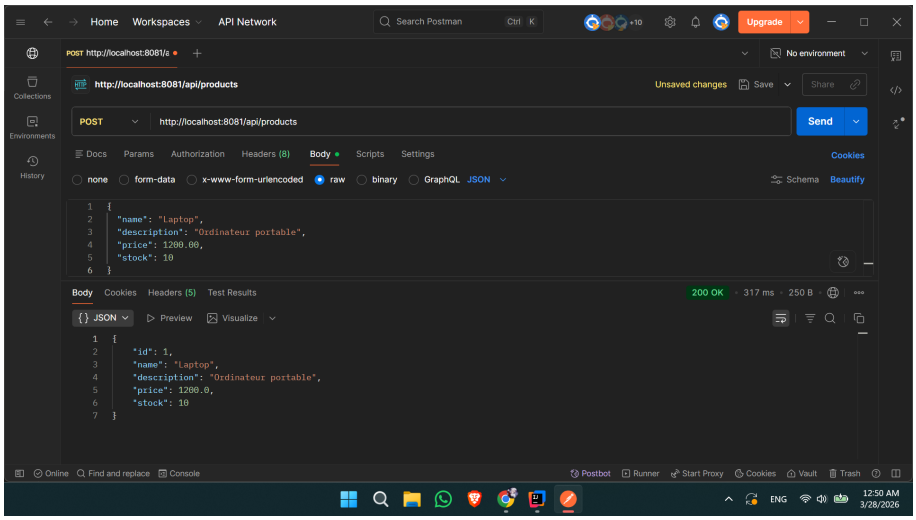
— Gestion robuste des erreurs et des cas d'indisponibilite.

Cette architecture favorise la **scalabilite**, la **maintenabilite** et la **resilience** du systeme, conformement aux bonnes pratiques du developpement logiciel distribue moderne.

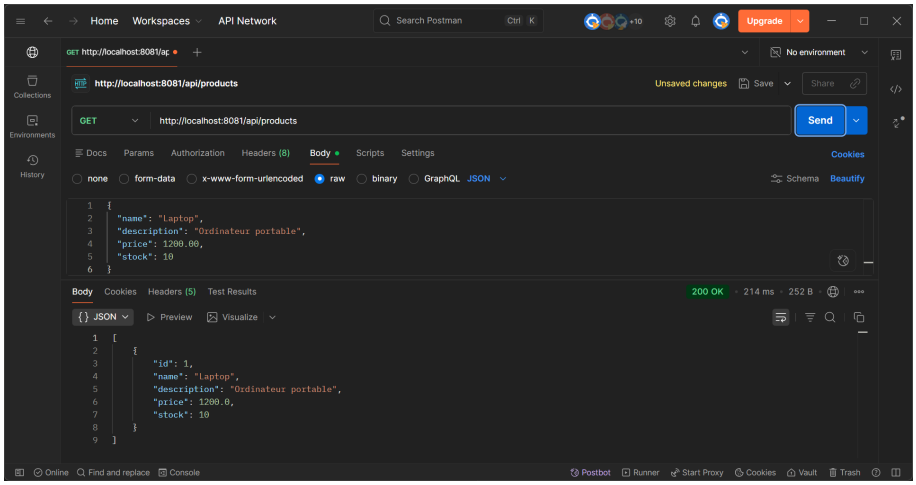
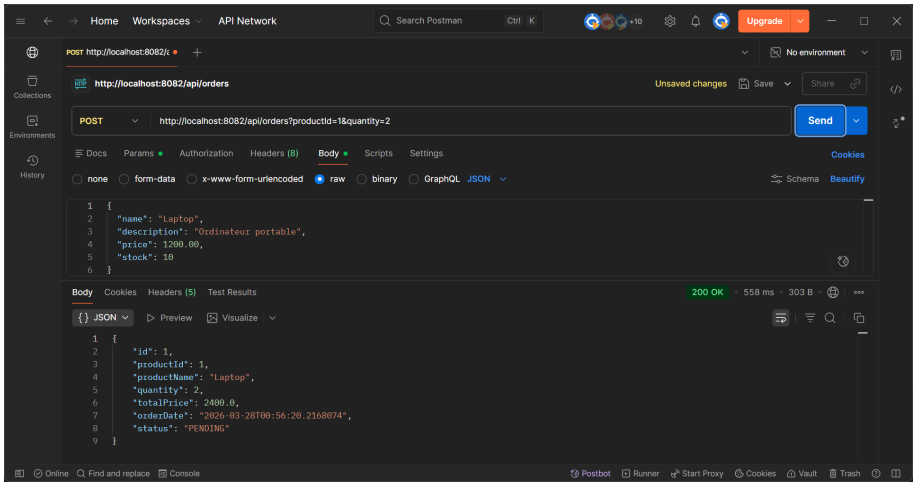
5.1 Tableau de bord Eureka Server



5.2 Test 1 — Creation d'un produit (POST)



5.3 Test 2 — Liste des produits (GET)



5.4 Test 3 Créer une commande (POST)